

1 Problem Definition and Approach

In this project, we address the problem of minimizing non-productive time during the drilling process, that consists in relocating the drill bit to a new position in order to drill another hole.

We develop two solution approaches:

- **Exact approach:** We formulated the problem as a mixed integer-linear programming (MILP) model and solved it using IBM CPLEX to obtain optimal solutions.
- **Heuristic approach:** We also designed a genetic algorithm (GA) implemented in C++. This approach aims to find an approximate solution within a smaller computational time compared to the exact approach.

Assumptions: We assume that the drilling sequence is predefined and all hole locations are known in advance. We further assume that the drill can move at constant speed in any direction, and that all holes are of the same size and can be drilled with the same drill bit. The repositioning time is determined solely by the (euclidean) travel distance of the drill bit.

Notice that the constant speed is an overly simplifying assumption for the real-world. The acceleration and deceleration of the tool are significant factors, especially when dealing with short distances [KHODABAKHSHI2021426].

2 Exact method

2.1 Mixed Integer-Linear Programming formulation

The following is the MILP formulation that has been implemented in order to solve exactly the symmetric TSP problem. It is inspired by formulation in [Gavish1978TheTS].

Sets:

- N : set of graph nodes, representing the holes.
- A : set of arcs (i, j) for all $i, j \in N$, representing the trajectory covered by the drill to move from hole i to hole j .

Parameters:

- c_{ij} : time taken by the drill to move from i to j , for all $(i, j) \in A$;
- 0: arbitrarily selected starting node, where $0 \in N$.

Decision Variables:

- x_{ij} : amount of the flow shipped from i to j , for all $(i, j) \in A$;
- y_{ij} : 1 if arc (i, j) ships some flow, 0 otherwise, for all $(i, j) \in A$.

Formulation:

$$\min \sum_{i,j:(i,j) \in A} c_{ij} y_{ij} \quad (1)$$

$$s.t. \sum_{i:(i,k) \in A} x_{ik} - \sum_{j:(k,j) \in A, j \neq 0} x_{kj} = 1 \quad \forall k \in N \quad (2)$$

$$\sum_{j:(i,j) \in A} y_{ij} = 1 \quad \forall i \in N \quad (3)$$

$$\sum_{i:(i,j) \in A} y_{ij} = 1 \quad \forall j \in N \quad (4)$$

$$x_{ij} \leq (|N| - 1) y_{ij} \quad \forall (i, j) \in A, j \neq 0 \quad (5)$$

$$x_{ij} \in \mathbb{R} \quad \forall (i, j) \in A, j \neq 0 \quad (6)$$

$$y_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \quad (7)$$

2.2 Implementation

Variables and objective function Let n be the number of nodes. The number e of undirected edges (excluding self-loops) is equivalent to $n * (n - 1)$. For our problem we declared e binary variables $y_0, \dots, y_{e-1}$ and $n * (n - 1)$ integer variables, $x_0, \dots, x_{e-1}$. Notice that every variable represent a directed arc that is not a self-loop, and viceversa every directed arc that is not a self-loop is represented by a variable x and a variable y .¹

The objective function is a sum of e variables, precisely $\sum_{i \in [0, e) \cup \mathbb{N}} y_i$, and our goal is to minimize it.

Rows We had to rewrite constraint 5 so to have a real number on the right-hand side:

$$x_{ij} - (|N| - 1) y_{ij} \leq 0 \quad \forall (i, j) \in A, j \neq 0$$

Let $\mathbf{u}$ be the vector $[y_0, \dots, y_{e-1}, x_0, \dots, x_{e-1}]$. Let F, G, H, L be matrices such that constraints 2–5 are equivalent to $F\mathbf{u} = 1, G\mathbf{u} = 1, H\mathbf{u} = 1, L\mathbf{u} \leq 0$

For every $\eta \in \{F, G, H, L\}$ we have defined the arrays ηsense , $\eta\text{rmatbeg}$, $\eta\text{rmatind}$, $\eta\text{rmatval}$, $\eta\text{rowname}$ and the integers ηrcnt and ηnzcnt to be passed to `CPXaddrows`. In this way, only four (checked) calls to `CPXaddrows` are performed to solve the mixed integer-linear problem.

Classes The program is contained almost entirely within the `main()` function. This style follows that of CPLEX library examples, which are typically written inside the `main()` procedure, and has been implemented because of the limited size of the codebase in term of single lines of code (*s.l.o.c.*) and the absence of need to separate logic into multiple classes.

Five additional classes are used:

- **TSPGraphReader**, responsible for reading and parsing a graph from file.
- **Graph**, which stores a the adjacency matrix of a graph and provides static method to convert from single coordinate to double coordinate.

¹In facts the class **Graph** defines a one-to-one mapping between integers $k \in [0, e) \cup \mathbb{N}$ and couples of integer (i, j) s.t. $i, j < n \wedge i \neq j$

- **Node**, that represents a two-dimensional point, to be used as a vertex of a **Graph**. Provides a method that returns the distance from another **Node**.
- **Path**, that represents an acyclic path in the graph.
- **CommandLineOptions**, which handles command-line arguments such as the input filename.

The header `cpxmacro.h`, that defines the macros `DECL_ENV`, `DECL_PROB`, and `CHECKED_CPX_CALL`, the array of character `errmsg` and the integer `status` has been included in our project.

We have used Valgrind Static Analyzer to check for memory leaks.

2.3 Handling the MIP Optimality Gap in CPLEX

In our experiments, for many problem instances, regardless of the computation time allocated to the solver, the termination status returned by CPLEX is `CPXMIP_OPTIMAL_TOL`, which indicates that an optimal solution has been found within the tolerance defined by either the relative or absolute MIP gap.

In solving mixed-integer programming (MIP) problems using CPLEX, one common approach to achieving near-exact optimality is to explicitly tighten the optimality gap by setting the parameter `CPXPARAM_MIP_Tolerances_MIPGap` to a sufficiently small value (e.g., 10^{-6}).

However, in our experiments, setting `CPXPARAM_MIP_Tolerances_MIPGap` to very small values resulted in memory corruption errors, such as `double free or corruption (!prev)`. Given such failures, manually tightening the MIP optimality gap was deemed infeasible. Hence the default value for `CPXPARAM_MIP_Tolerances_MIPGap` (i.e. 10^{-3}) has been left unchanged.